

Zeta – Technical Documentation

Application

Name: Zeta

StudentName: José Barros González

Programme:DAM (Multiplatform Application Development)

Course: 2nd Year

Table of Contents

1. System Architecture
 2. API
 3. Database
 4. Web Application
 5. Mobile Application
 6. Desktop Application
 7. Security
 8. Deployment
 9. Testing
 - 10.User Guide and Maintenance
 - 11.Future Improvements
-

1. System Architecture

1.1 Overview

The Zeta system is designed following a **client-server architecture** where multiple frontends consume a centralized REST API.

The system is composed of:

- A **REST API (Java)** responsible for business logic
- A **MariaDB database** for data persistence
- Three clients:
 - Web application (PHP)
 - Mobile application (Android)
 - Desktop application (planned)

All communication is done using **HTTP requests with JSON format**.

1.2 Architecture Flow

Clients send requests → API processes logic → Database stores/retrieves data → API returns JSON → Clients render UI.

1.3 Technologies Used

Layer	Technology
Backend	Java (JAX-RS, JDBC)
Database	MariaDB
Mobile	Android (Java)
Web	PHP (MVC + cURL)
Communication	HTTP + JSON

2. API

2.1 Base URL

<http://localhost:8080/xapi/rest>

2.2 Endpoints

Users

Method	Endpoint	Description
POST	/user	Create user
POST	/user/login	Login
GET	/user/{email}	Get user by email
DELETE	/user/{id}	Delete user

Tweets

Method	Endpoint	Description
GET	/tweet	Get timeline
POST	/tweet	Create tweet
DELETE	/tweet/{id}	Delete tweet

2.3 Authentication

The system uses a **basic authentication model**:

- Users log in with username and password
- Session data is stored in clients (SharedPreferences or PHP sessions)

2.4 Example Requests

Create User

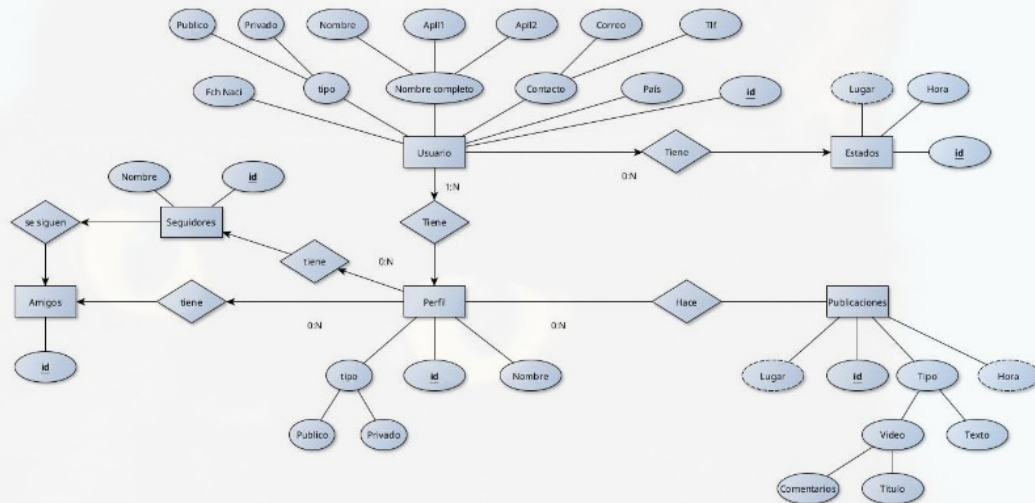
```
{
  "username": "meu",
  "email": "meu@mail.com",
  "password": "1234",
  "display_name": "Meu"
}
```

Create Tweet

```
{
  "username": "meu",
  "handle": "@meu",
  "content": "Hello world"
}
```

3. Database

MODELO RELACIONAL



3.1 Tables

- users
- tweets
- follows
- likes
- posts
- daily_loot

3.2 Main Tables

users

Field	Type	Description
id	INT	Primary key
username	VARCHAR	Unique username
email	VARCHAR	User email
password_hash	VARCHAR	Password
display_name	VARCHAR	Visible name

tweets

Field	Type	Description
id	INT	Primary key
username	VARCHAR	Owner
handle	VARCHAR	User handle
content	TEXT	Tweet text

3.3 Relationships

- One user → many tweets
- One user → many likes
- One user → many follows

3.4 Rules

- Tweets cannot be empty
 - Only owners can delete tweets
-

4. Web Application

4.1 Architecture

The web application follows an MVC pattern:

- Controllers (logic)
- Models (API communication)
- Views (UI)

4.2 API Connection

Uses cURL to send requests:

- GET /tweet
- POST /tweet
- POST /user/login

4.3 Features

- Login / Register
- Timeline view
- Create tweet
- Delete tweet

5. Mobile Application

5.1 Structure

Main components:

- MainActivity (login)
- IniciarSesion (register)
- Principal (timeline)
- PerfilActivity (profile)

5.2 Features

- Login & session storage
- Timeline loading
- Create tweet
- Delete tweet
- Delete account

5.3 API Communication

Uses HttpURLConnection and JSON parsing.

6. Desktop Application

6.1 Technology

The desktop application is developed using **C# with .NET (Windows Forms)**.

It acts as a third client that interacts directly with the REST API, similar to the mobile and web applications.

6.2 Architecture

The application follows a simple UI-driven architecture:

- Forms (UI layer)
- Event handlers (logic layer)
- API calls (data layer)

User actions (login, post, delete) trigger HTTP requests to the backend API.

6.3 Features

- User login
- Display of tweets
- Create new tweets
- Delete tweets (only if owned by the user)

6.4 API Communication

The desktop application communicates with the API using HTTP requests, following the same endpoints as other clients:

- POST /user/login
- GET /tweet
- POST /tweet
- DELETE /tweet/{id}

6.5 Observations

Although functional, the desktop application:

- Uses a basic architecture without clear separation of concerns
 - Lacks advanced error handling
 - Shares the same authentication limitations as other clients
-

7. Security

7.1 Implemented

- PreparedStatement (SQL injection protection)
- Session management

7.2 Weaknesses

- Passwords not encrypted
- No JWT or tokens
- No role system

7.3 Improvements

- Add password hashing
 - Implement authentication tokens
-

8. Deployment

8.1 API

- Deploy on Tomcat
- Connect to MariaDB

8.2 Web

- Deploy on Apache
- Configure API URL

8.3 Mobile

- Run via Android Studio
-

10. User Guide

Steps:

1. Start database
 2. Run API
 3. Open web or mobile app
-

11. Future Improvements

- JWT authentication
- Follow system
- Like system
- Notifications
- UI improvements